

Item cold-start: recommending StackExchange questions

In this example we'll use the StackExchange dataset to explore recommendations under item-cold start. Data dumps from the StackExchange network are available at <https://archive.org/details/stackexchange>, and we'll use one of them — for stats.stackexchange.com — here.

The consists of users answering questions: in the user-item interaction matrix, each user is a row, and each question is a column. Based on which users answered which questions in the training set, we'll try to recommend new questions in the training set.

Let's start by loading the data. We'll use the `datasets` module.

```
import numpy as np

from lightfm.datasets import fetch_stackexchange

data = fetch_stackexchange('crossvalidated',
                           test_set_fraction=0.1,
                           indicator_features=False,
                           tag_features=True)

train = data['train']
test = data['test']
```

Let's examine the data:

```
print('The dataset has %s users and %s items, '
      'with %s interactions in the test and %s interactions in the training '
      'set.'
      % (train.shape[0], train.shape[1], test.getnnz(), train.getnnz()))
```

The dataset has 3221 users and 72360 items, with 4307 interactions in the test and 57830 interactions in the training set.

The training and test set are divided chronologically: the test set contains the 10% of interactions that happened after the 90% in the training set. This means that many of the questions in the test set have no interactions. This is an accurate description of a questions answering system: it is most important to recommend questions that have not yet been answered to the expert users who can answer them.

A pure collaborative filtering model

This is clearly a cold-start scenario, and so we can expect a traditional collaborative filtering model to do very poorly. Let's check if that's the case:

```
# Import the model
from lightfm import LightFM

# Set the number of threads; you can increase this
# if you have more physical cores available.
NUM_THREADS = 2
NUM_COMPONENTS = 30
NUM_EPOCHS = 3
ITEM_ALPHA = 1e-6

# Let's fit a WARP model: these generally have the best performance.
model = LightFM(loss='warp',
                item_alpha=ITEM_ALPHA,
                no_components=NUM_COMPONENTS)

# Run 3 epochs and time it.
%time model = model.fit(train, epochs=NUM_EPOCHS, num_threads=NUM_THREADS)
```

```
CPU times: user 12.9 s, sys: 8 ms, total: 12.9 s
Wall time: 6.52 s
```

As a means of sanity checking, let's calculate the model's AUC on the training set first. If it's reasonably high, we can be sure that the model is not doing anything stupid and is fitting the training data well.

```
# Import the evaluation routines
from lightfm.evaluation import auc_score

# Compute and print the AUC score
train_auc = auc_score(model, train, num_threads=NUM_THREADS).mean()
print('Collaborative filtering train AUC: %s' % train_auc)
```

```
Collaborative filtering train AUC: 0.887519
```

Fantastic, the model is fitting the training set well. But what about the test set?

```
# We pass in the train interactions to exclude them from predictions.  
# This is to simulate a recommender system where we do not  
# re-recommend things the user has already interacted with in the train  
# set.  
test_auc = auc_score(model, test, train_interactions=train,  
num_threads=NUM_THREADS).mean()  
print('Collaborative filtering test AUC: %s' % test_auc)
```

Collaborative filtering test AUC: 0.34728

This is terrible: we do worse than random! This is not very surprising: as there is no training data for the majority of the test questions, the model cannot compute reasonable representations of the test set items.

The fact that we score them lower than other items ($AUC < 0.5$) is due to estimated per-item biases, which can be confirmed by setting them to zero and re-evaluating the model.

```
# Set biases to zero  
model.item_biases *= 0.0  
  
test_auc = auc_score(model, test, train_interactions=train,  
num_threads=NUM_THREADS, check_intersections=False).mean()  
print('Collaborative filtering test AUC: %s' % test_auc)
```

Collaborative filtering test AUC: 0.496266

A hybrid model

We can do much better by employing LightFM's hybrid model capabilities. The StackExchange data comes with content information in the form of tags users apply to their questions:

```
item_features = data['item_features']
tag_labels = data['item_feature_labels']

print('There are %s distinct tags, with values like %s.' %
      (item_features.shape[1], tag_labels[:3].tolist()))
```

```
There are 1246 distinct tags, with values like [u'bayesian', u'prior',
u'elicitation'].
```

We can use these features (instead of an identity feature matrix like in a pure CF model) to estimate a model which will generalize better to unseen examples: it will simply use its representations of item features to infer representations of previously unseen questions.

Let's go ahead and fit a model of this type.

```
# Define a new model instance
model = LightFM(loss='warp',
                item_alpha=ITEM_ALPHA,
                no_components=NUM_COMPONENTS)

# Fit the hybrid model. Note that this time, we pass
# in the item features matrix.
model = model.fit(train,
                  item_features=item_features,
                  epochs=NUM_EPOCHS,
                  num_threads=NUM_THREADS)
```

As before, let's sanity check the model on the training set.

```
# Don't forget the pass in the item features again!
train_auc = auc_score(model,
                      train,
                      item_features=item_features,
                      num_threads=NUM_THREADS).mean()
print('Hybrid training set AUC: %s' % train_auc)
```

```
Hybrid training set AUC: 0.86049
```

Note that the training set AUC is lower than in a pure CF model. This is fine: by using a lower-rank item feature matrix, we have effectively regularized the model, giving it less freedom to fit the training data.

Despite this the model does much better on the test set:

```
test_auc = auc_score(model,
                      test,
                      train_interactions=train,
                      item_features=item_features,
                      num_threads=NUM_THREADS,
                      check_intersections=False).mean()
print('Hybrid test set AUC: %s' % test_auc)
```

```
Hybrid test set AUC: 0.703039
```

This is as expected: because items in the test set share tags with items in the training set, we can provide better test set recommendations by using the tag representations learned from training.

Bonus: tag embeddings

One of the nice properties of the hybrid model is that the estimated tag embeddings capture semantic characteristics of the tags. Like the word2vec model, we can use this property to explore semantic tag similarity:

```
def get_similar_tags(model, tag_id):
    # Define similarity as the cosine of the angle
    # between the tag latent vectors

    # Normalize the vectors to unit length
    tag_embeddings = (model.item_embeddings.T
                     / np.linalg.norm(model.item_embeddings, axis=1)).T

    query_embedding = tag_embeddings[tag_id]
    similarity = np.dot(tag_embeddings, query_embedding)
    most_similar = np.argsort(-similarity)[1:4]

    return most_similar

for tag in (u'bayesian', u'regression', u'survival'):
    tag_id = tag_labels.tolist().index(tag)
    print('Most similar tags for %s: %s' % (tag_labels[tag_id],
                                           tag_labels[get_similar_tags(model,
                                           tag_id)]))
```

```
Most similar tags for bayesian: [u'posterior' u'mcmc' u'bayes']  
Most similar tags for regression: [u'multicollinearity' u'steppwise-regression'  
u'multiple-regression']  
Most similar tags for survival: [u'cox-model' u'kaplan-meier' u'odds-ratio']
```